

Assignment #1



Design and Analysis of Algorithm

SUBMISSION GUIDELINES:

1. The assignment must be submitted within the due date.
 2. The assignment must be on paper and in handwritten form.
 3. Plagiarism is not allowed. In the case of a pledge, you will score zero marks.
 4. Do not submit your assignment after the deadline.
 5. Do not copy from any other source.
 6. It is an individual submission.
 7. The deadline for this assignment is. **22-02-2024 Thursday in class of Week-05-Lecture-02.**
 8. Questions should be written with answers.
 9. **Submit 16 pattern sheets for Q#1 (mandatory) handwritten before class (Week-05-Lecture-02)**
-

Question no 1:

The following are some pieces of code. Provide a Dry Run of all samples of 5 Passes. Also, specify its worst-case time complexity.

Must Follow the Pattern in the attached File, Otherwise Marks will be Zero.

A

```
for ( i=0 ; i<n ; i++ )
    for( j=0 ; j<n ; j++ )
    {
        sum[i] += entry[i][j];
    }
```

B

```
for ( i=0 ; i<n ; i++ )
    for( j=0 ; j<i ; j++ )
        m += j;
```

C

```
i = 1;
while (i < n) {
    tot += i;
    i = i * 2;
}
```

D

```
i = n;
while (i > 0) {
    tot += i;
    i = i / 2;
}
```

E

```
for ( i=0 ; i<n ; i++ )
    for( j=0 ; j<n ; j++ )
        sum[i] += entry[i][j][0];
```

F

```
for ( i=0 ; i<n ; i++ )
    for( j=0 ; j<n ; j++ )
        for( k=0 ; k<n ; k++ )
            sum[i][j] += entry[i][j][k];
```

G

```
for ( i=0 ; i<n ; i++ )
    for( k=0 ; k<n ; k++ )
        sum[i] += entry[i][0][k];
```

H

```
for ( i=0 ; i<n ; i++ )
    for( j=0 ; j< sqrt(n) ; j++ )
        m += j;
```

I

```
for ( i=0 ; i<n ; i++ )
{
    m += j;
    m += j;
    m += j;
    ...
    m += j;    // 31 times
}
```

J

```
int total(int n)
    for( i=0 ; i < n; i++)
        subtotal += i;
main()
    for ( i=0 ; i<n ; i++ )
        tot += total(i);
```

K

```
for ( i=0 ; i<n ; i++ ) {
    subtotal = 0;
    for( j=0 ; j < i; j++)
        subtotal += j;
    tot += subtotal;
}
```

L

```
int sum = 0;
for (int i = 1; i
= n; i++) {
    for (int j = 1; j
<= i; j++) {
        sum += 1;
```

```
}  
}
```

M

```
int sum = 0;  
for (int i = 0; i < n; ++i) {  
    for (int j = i; j <= 2*i; ++j) {  
        sum += 1;  
    }  
}
```

N

```
int sum = 0;  
for (int i =  
1; i <= n;  
i*= 2) {  
    for (int j = 0; j < i; j++) {  
        sum += 1;  
    }  
}
```

Question no 3:

1. Simplify the time complexity expression.

O

```
int sum = 0, i = 1;  
while (i <= n) {  
    for (int j = 0; j < i; j++) {  
        sum += 1;  
    }  
    i *= 2;  
}
```

P

```
int sum = 0
for (int i = 0; i < n; i++) {
    for (int j = i; j < n; j++) {
        for (int k = j; k < n; k += j) {
            sum += 1;
        }
    }
}
```

Question no 2:

Case A:

Imagine you're a librarian, and you have a small number of books that are almost in order, but one book is out of place.

This is like the list [9, 1, 2, 3, 4, 5, 6, 7, 8]

Case B:

Imagine you're working for a music streaming service, and you need to sort a user's recently played songs based on their last playback time. The list isn't very long (around 20-30 songs), but you want to quickly display the songs in chronological order with the most recently played song appearing first.

- In the case **A** and **B** which Sorting Algorithm (Insertion sort, Selection sort, bubble sort) is best. Explain? And explain why other sorting algorithms are not.
- Write time complexity of each Sorting Algorithms (All Cases).

Question no 3:

A

Apply Insertion Sort, Bubble Sort, Selection Sort on **Your Name** e.g. (ahmad → aadhm)
Provide Dry Run.

B

Is there Sorting Algorithms specify which properties hold which are given below:

1. In-place, online, Stable Sorting

